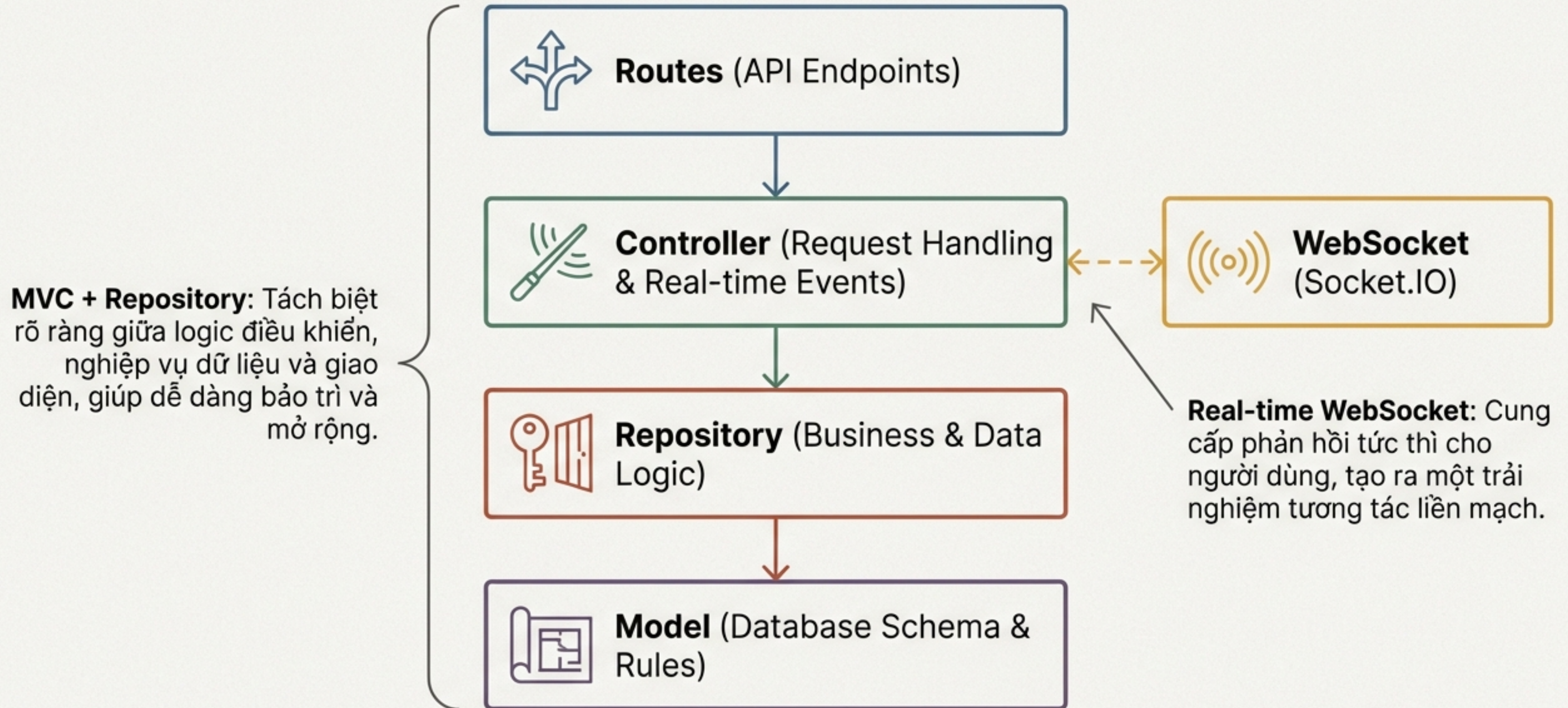


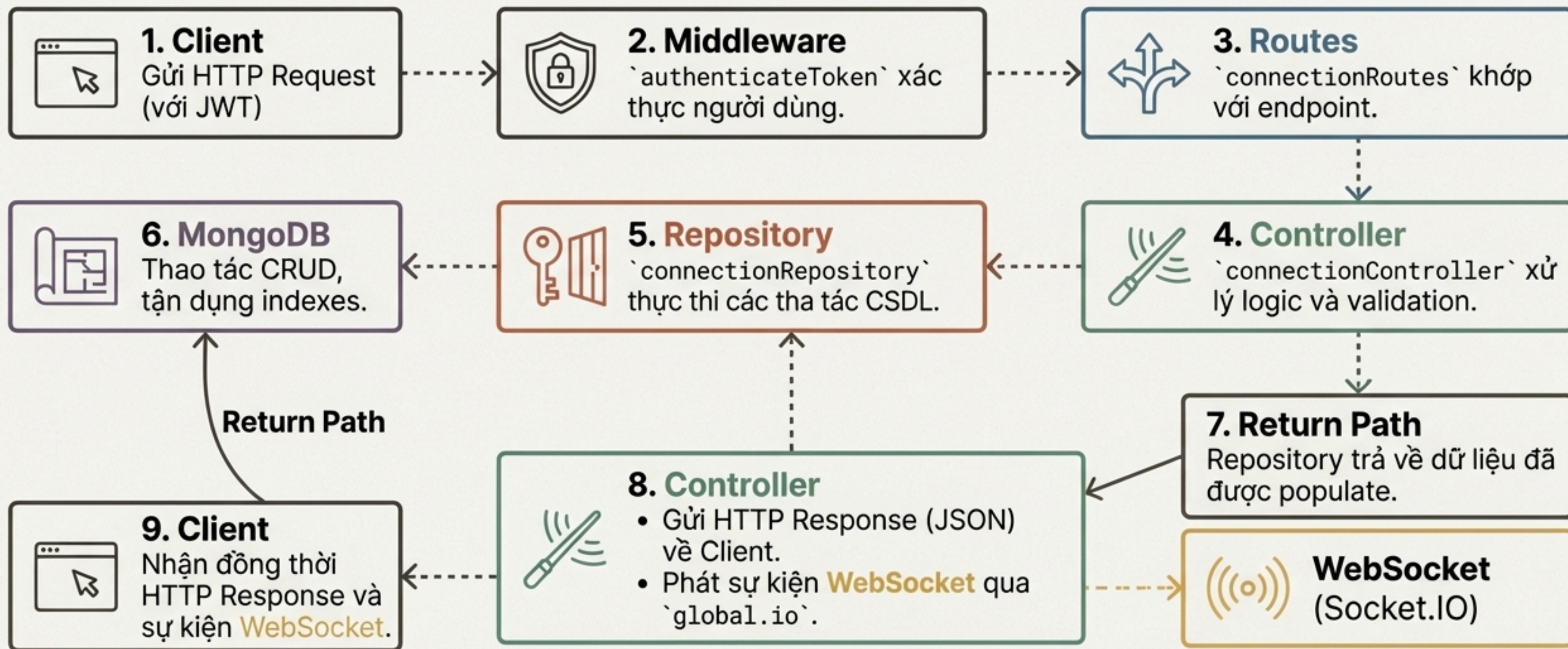
Xây Dựng Nền Tảng Cho Sự Kết Nối

Phân tích Kiến trúc Hệ thống Quản lý Bạn bè
(Connection Flow)

Sơ Đồ Kiến Trúc Tổng Thể: Một Hệ Thống Bền Vững



Hành Trình Của Một Yêu Cầu: Từ Giao Diện Đến Cơ Sở Dữ Liệu



Mỗi yêu cầu được xử lý qua một chuỗi các lớp được xác định rõ ràng, đảm bảo tính nhất quán và bảo mật.

Nền Tảng Dữ Liệu: Định Nghĩa Kết Nối với Mongoose Schema



```
// backend/src/models/Connection.js
const connectionSchema = new mongoose.Schema({
  requester: { type: ObjectId, ref: 'User',
    required: true, index: true },
  recipient: { type: ObjectId, ref: 'User',
    required: true, index: true },
  status: {
    type: String,
    enum: ['pending', 'accepted', 'rejected',
'blocked'],
    default: 'pending',
    index: true
  },
  blockedBy: { type: ObjectId, ref: 'User' },
}, { timestamps: true });
```

- **`requester` & `recipient`**: Xác định hai người dùng trong mối quan hệ. Luôn là ObjectId tham chiếu đến collection 'User'.
- **`status`**: Trạng thái của kết nối, được kiểm soát chặt chẽ bởi Enum để đảm bảo tính toàn vẹn của state machine.
- **`blockedBy`**: Lưu trữ ID của người chủ động chặn, cần thiết cho logic ủy quyền khi bỏ chặn.
- **`timestamps`**: Tự động ghi lại thời gian tạo và cập nhật, hữu ích cho việc sắp xếp và theo dõi.

Trí Tuệ Tích Hợp: Tối Ưu Hóa Hiệu Năng và Đảm Bảo Toàn Vẹn

Indexing Strategy - Chìa Khóa Tốc Độ Truy Vấn



- **Compound Index:**

`{requester: 1, recipient: 1}` với `unique: true`

Mục đích: Đảm bảo không có kết nối trùng lặp giữa hai người dùng và cho phép tra cứu mỗi quan hệ $O(1)$.



- **Single Indexes:**

`{requester: 1, status: 1}` và `{recipient: 1, status: 1}`

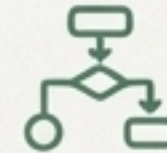
Mục đích: Tăng tốc đáng kể các truy vấn phổ biến như "lấy danh sách bạn bè" hoặc "lấy lời mời đang chờ".

Logic Tự Bảo Vệ - Ngăn Chặn Dữ Liệu Sai



- **Pre-save Hook:**

Tự động kiểm tra và ngăn người dùng tự kết bạn với chính mình (``requester === recipient``).



- **Enum `status`:**

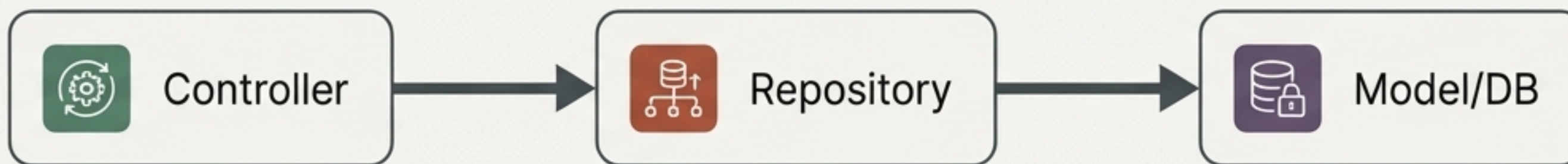
Buộc trạng thái phải là một trong các giá trị hợp lệ, loại bỏ các trạng thái không mong muốn.

Quyết định Thiết kế Trọng tâm

Toàn vẹn dữ liệu được ưu tiên hàng đầu. Các ràng buộc và logic được tích hợp ngay tại tầng Model để ngăn chặn dữ liệu không hợp lệ trước khi nó được lưu trữ.

Lớp Cổng Gác: Repository Pattern là Nơi Logic Dữ Liệu Tọa Lạc

Tất cả logic nghiệp vụ liên quan đến dữ liệu kết nối—từ tạo yêu cầu đến các truy vấn phức tạp—đều được đóng gói hoàn toàn trong `connectionRepository.js`.



Các Trách Nhiệm Chính:

-  **Abstraction:** Che giấu sự phức tạp của các truy vấn Mongoose.
-  **Single Source of Truth:** Là điểm truy cập duy nhất vào dữ liệu kết nối, đảm bảo logic nhất quán.
-  **Data Shaping:** Luôn populate thông tin người dùng cần thiết, giảm số lượng API calls từ phía client.
-  **Error Handling:** Ném ra các lỗi nghiệp vụ có ý nghĩa (ví dụ: 'Friend request already sent').

Logic Tinh Gọn: Giải Quyết Các Bài Toán Phức Tạp

The Bidirectional Query (Truy vấn hai chiều)

Kiểm tra một kết nối đã tồn tại giữa User A và B, bất kể ai là người gửi.

```
// connectionRepository.js
const existingConnection = await Connection.findOne({
  $or: [
    { requester: requesterId, recipient: recipientId },
    { requester: recipientId, recipient: requesterId },
  ],
});
```

Sử dụng toán tử `\$or` để kiểm tra cả hai chiều trong một truy vấn duy nhất, đảm bảo tính hiệu quả và chính xác.

The Suggestion Engine (Gợi ý bạn bè)

Tìm “bạn của bạn” mà bạn chưa kết nối.

Sử dụng MongoDB Aggregation Pipeline với các stage `\$lookup` và `\$match`.

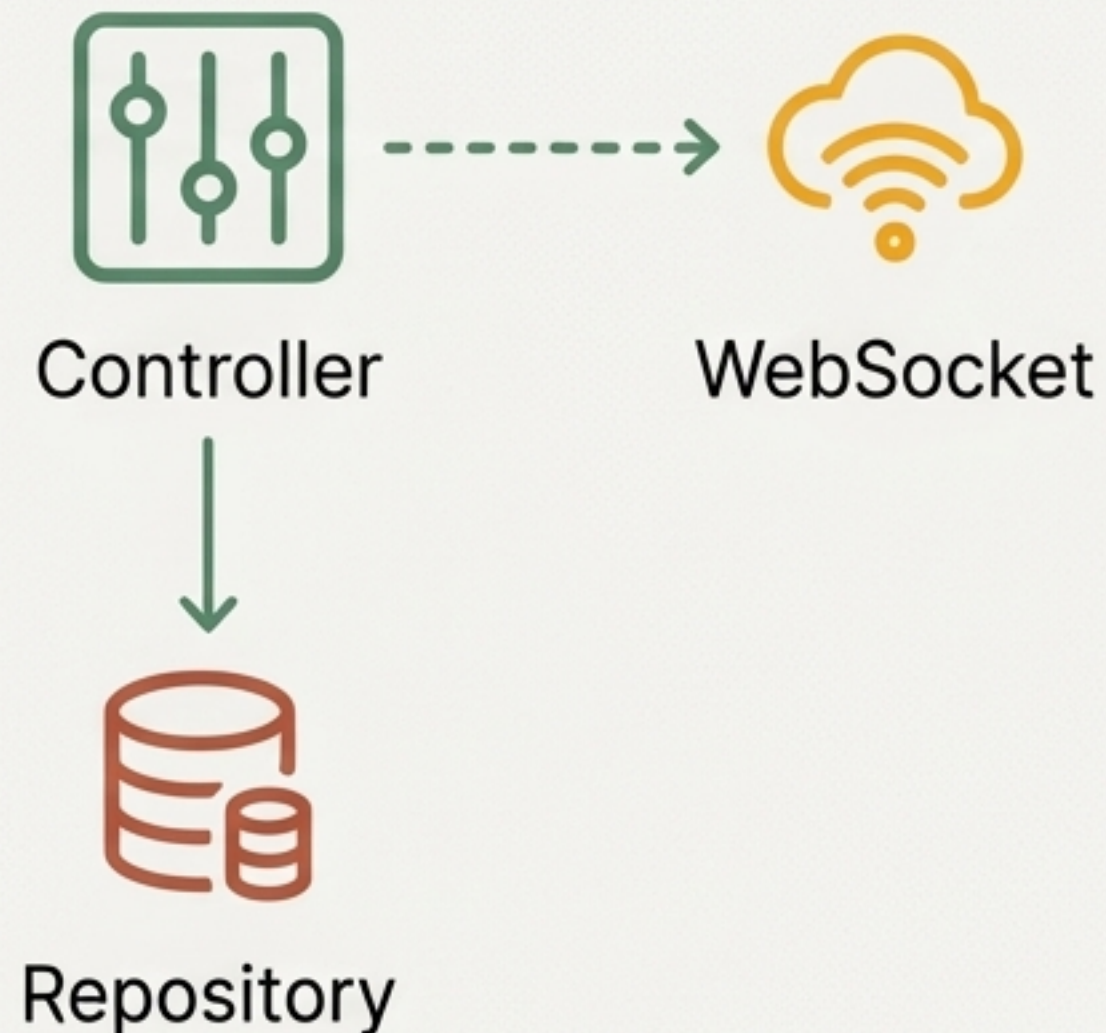
Một giải pháp tối ưu, thực hiện các phép tính phức tạp ngay trên server CSDL thay vì xử lý nhiều truy vấn và lặp qua dữ liệu ở tầng ứng dụng.

Quyết định Thiết kế Trọng tâm

Logic phức tạp được xử lý càng gần nguồn dữ liệu càng tốt. Điều này giảm thiểu độ trễ mạng và tối ưu hóa việc sử dụng tài nguyên của ứng dụng.

Lớp Điều Phối: Controller Xử Lý Yêu Cầu và Kích Hoạt Sự Kiện Real-time

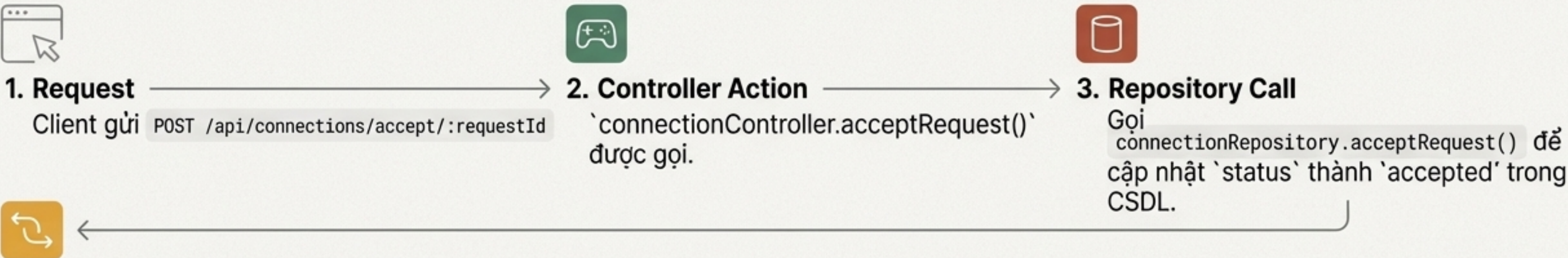
`connectionController.js` là cầu nối giữa các yêu cầu HTTP từ client và logic nghiệp vụ trong Repository.



Các Trách Nhiệm Chính:

1. **Request Handling:** Tiếp nhận các request, xác thực input (ví dụ: `recipientId` là bắt buộc).
2. **Authorization:** Phối hợp với Repository để kiểm tra quyền hạn (ví dụ: chỉ người nhận mới có thể chấp nhận lời mời).
3. **Orchestration:** Gọi các phương thức Repository thích hợp để thực thi hành động.
4. **Response Formatting:** Định dạng và gửi lại phản hồi JSON nhất quán cho client.
5. **Real-time Integration:** Truy cập `global.io` để phát các sự kiện Socket.IO đến những người dùng liên quan.

Case Study: Luồng Chấp Nhận Lời Mời và Đồng Bộ Hóa Tức Thì



4. WebSocket Emission
Sau khi cập nhật thành công, Controller phát hai sự kiện riêng biệt:

→ **Tới người gửi (Requester):** `io.to('user-<requesterId>').emit('friend-request-accepted', ...)`
Mục đích: "Thông báo cho người gửi rằng lời mời của họ đã được chấp nhận. UI của họ có thể cập nhật ngay lập tức."

→ **Tới người nhận (Recipient):**
`io.to('user-<recipientId>').emit('friend-request-accepted', ...)`
Mục đích: "Cập nhật UI của người nhận, chuyển lời mời từ 'đang chờ' sang danh sách bạn bè mới."

Quyết định Thiết kế Trọng tâm
Sự kiện WebSocket không chỉ được phát đi mà còn được 'nhắm mục tiêu' (targeted) đến các phòng (rooms) cá nhân. Điều này đảm bảo thông tin chỉ được gửi đến những người dùng cần nó, tối ưu hóa băng thông và bảo mật.

Cổng Vào Hệ Thống: Thiết Kế API RESTful và Xác Thực Toàn Cục

Các API Endpoint

POST /api/connections/send

POST /api/connections/accept/:requestId

GET /api/connections/friends

DELETE /api/connections/unfriend/:friendId

POST /api/connections/block/:targetUserId

GET /api/connections/suggestions

GET /api/connections/status/:targetUserId

...và các route khác

Nguyên Tắc Thiết Kế Chính

RESTful Design: Sử dụng các phương thức HTTP (GET, POST, DELETE) một cách ngữ nghĩa để thể hiện rõ hành động trên tài nguyên.

Clear URL Patterns: Sử dụng URL parameters (ví dụ: `:requestId`) để xác định các tài nguyên cụ thể.

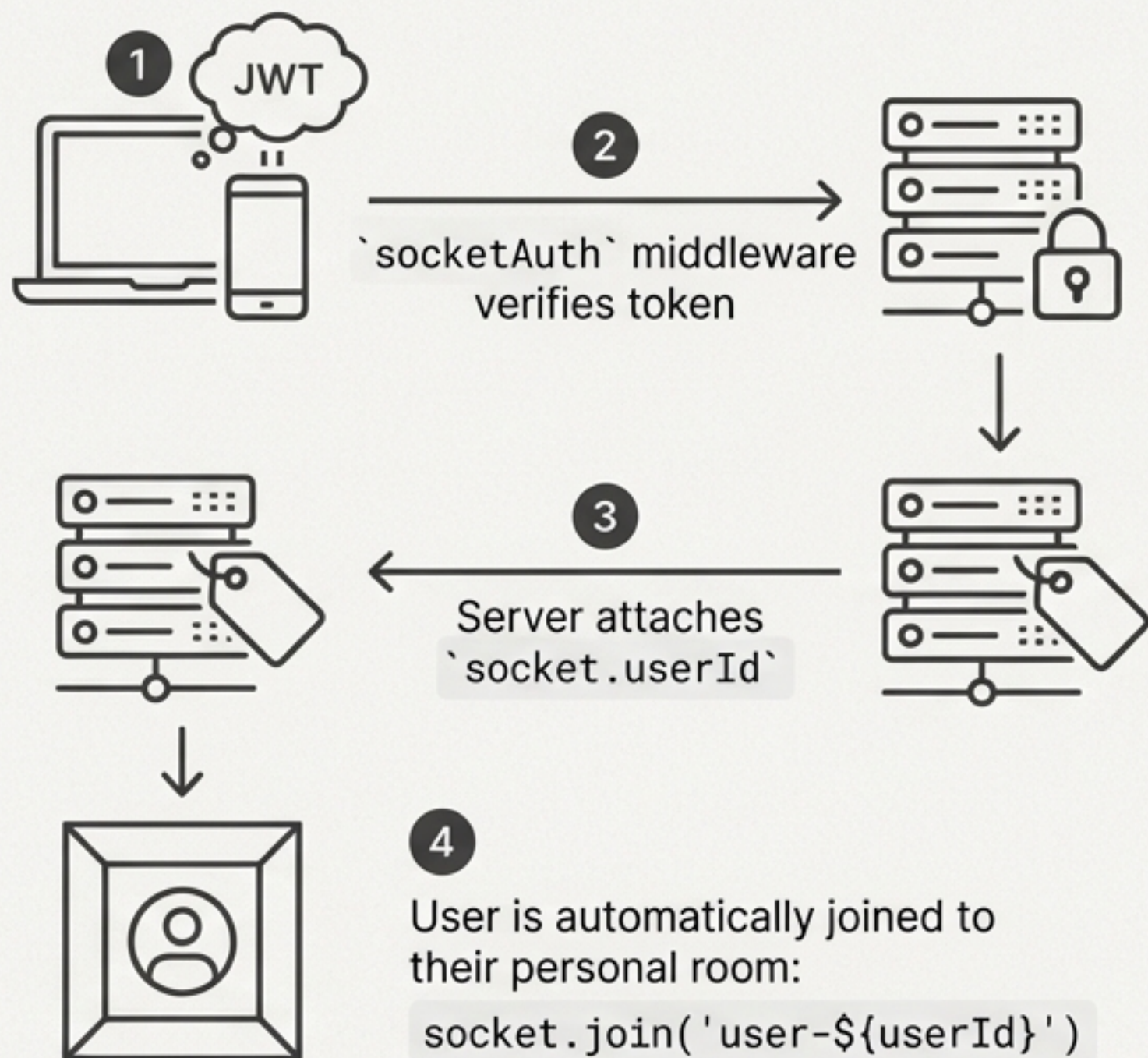
Global Authentication Middleware:

```
router.use(authenticateToken)
```

Tất cả các endpoint trong luồng Connection đều được bảo vệ. Mọi yêu cầu không có JWT hợp lệ sẽ bị từ chối trước khi đến được Controller, tạo ra một lớp bảo vệ vững chắc.



Hệ Thống Thần Kinh Real-time: Kiến Trúc Socket.IO



Core Strategy: Personal Rooms

- **Concept:** Mỗi người dùng khi kết nối sẽ tự động tham gia vào một "phòng" riêng tư có tên duy nhất (ví dụ: ``user-123``).
- **Benefit:** Cho phép gửi thông báo một cách có chủ đích và hiệu quả bằng `io.to('user-123').emit(...)` thay vì phát sóng (broadcast) cho tất cả mọi người. Điều này cực kỳ quan trọng cho hiệu năng và bảo mật.

Key Real-time Events

- `friend-request-received`
- `friend-request-accepted`
- `friend-request-cancelled`
- `unfriended`
- `user-blocked`

Trụ Cột I: Hiệu Năng Được Tối Ưu Hóa Từ Thiết Kế

Hiệu năng không phải là một tính năng bổ sung, mà là một nguyên tắc cốt lõi được áp dụng ở mọi tầng kiến trúc.



Database Level

- **Compound & Single Indexes:** Giảm thời gian truy vấn từ $O(n)$ xuống $O(\log n)$ hoặc $O(1)$ cho các thao tác tìm kiếm phổ biến.
- **Selective Population:** Chỉ lấy các trường cần thiết (``firstName``, ``avatar``, ...) khi populate, giảm lượng dữ liệu truyền tải.



Application Level

- **Aggregation Pipeline:** Đẩy các tính toán phức tạp (gợi ý bạn bè) về phía CSDL để xử lý hiệu quả hơn.
- **Lean Queries:** Sử dụng `.lean()` cho các truy vấn chỉ đọc để bỏ qua overhead của việc tạo Mongoose document.



Real-time Level

- **Room-based Targeting:** Tránh broadcast toàn hệ thống, chỉ gửi dữ liệu đến các client thực sự cần nó.

Trụ Cột II: Bảo Mật Toàn Diện và Toàn Vẹn Dữ Liệu

An toàn và bảo mật được đảm bảo thông qua một chiến lược phòng thủ theo chiều sâu (defense-in-depth).



- **Authentication (Xác thực)**
 - **API:** Mọi endpoint đều yêu cầu JWT hợp lệ thông qua ``authenticateToken`` middleware.
 - **WebSocket:** Middleware ``socketAuth`` xác thực token ngay khi kết nối được thiết lập.
- **Authorization (Ủy quyền)**
 - **Logic Checks:** Các kiểm tra chặt chẽ được thực hiện trong Repository và Controller. Ví dụ: chỉ ``requester`` mới có thể hủy yêu cầu, chỉ ``recipient`` mới có thể chấp nhận.
- **Data Validation & Integrity (Toàn vẹn dữ liệu)**
 - **Model Level:** Ngăn chặn tự kết bạn (``pre-save hook``).
 - **Database Level:** Ràng buộc ``unique`` trên compound index ngăn chặn các yêu cầu kết bạn trùng lặp.

Trụ Cột III: Nền Tảng Cho Tương Lai - Mở Rộng và Bảo Trì

Kiến trúc hiện tại không chỉ giải quyết các vấn đề của hôm nay mà còn tạo đà cho sự phát triển trong tương lai.



Scalability Path

- **Hạn chế hiện tại:** Socket.IO rooms trong bộ nhớ không thể mở rộng theo chiều ngang (horizontally).
- **Giải pháp đề xuất:** Tích hợp Redis adapter cho Socket.IO để chia sẻ trạng thái giữa nhiều server.
- **Cải tiến khác:** Phân trang (pagination) cho danh sách bạn bè, caching cho gợi ý, sử dụng background job cho các tác vụ nặng.

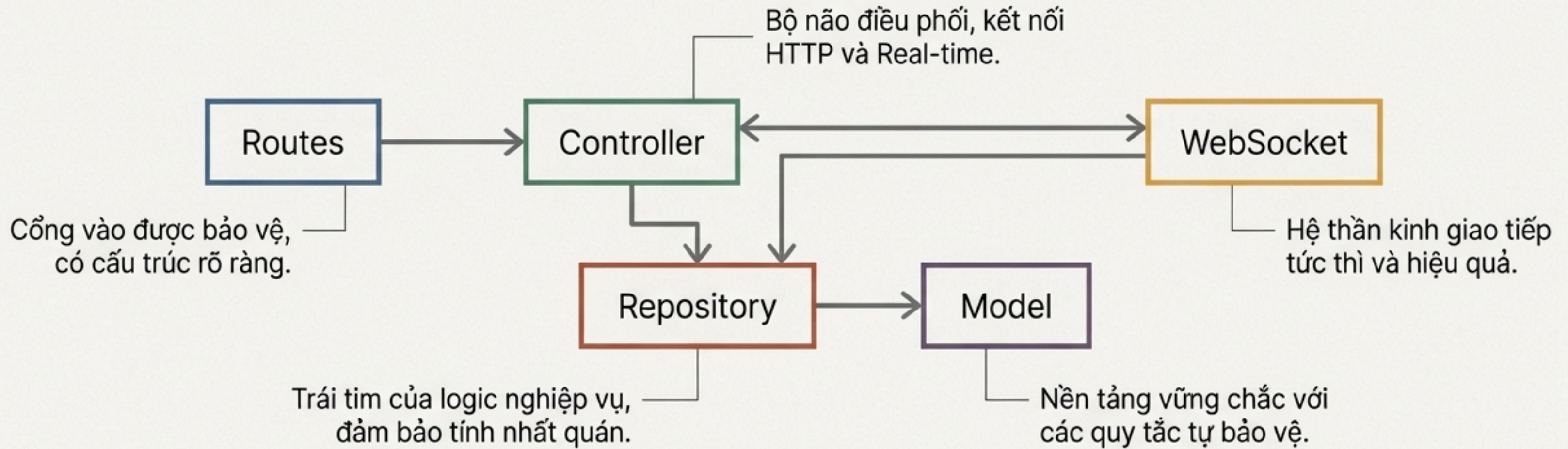
Robustness & Maintenance

- **Error Handling:** Sử dụng try-catch và định dạng response lỗi nhất quán để dễ dàng debug.
- **Testing Strategy:** Kiến trúc phân lớp cho phép Unit Test (Repository, Controller) và Integration Test (toàn bộ luồng API) một cách độc lập và hiệu quả.

Logging

- Ghi lại các ngoại lệ và sự kiện quan trọng, với đề xuất tích hợp các dịch vụ như Sentry để theo dõi lỗi tập trung.

Bức Tranh Toàn Cảnh: Một Kiến Trúc Được Xây Dựng Để Bền Vững



Hệ thống Connection không chỉ là một tập hợp các chức năng, mà là một kiến trúc được thiết kế có chủ đích—ưu tiên hiệu năng, bảo mật và khả năng mở rộng—tạo ra một nền tảng vững chắc cho các tương tác xã hội trong tương lai.